PROJECT REPORT

Titanic Survival Prediction

Using Machine Learning

Submitted by

Vikky Kumar

B.Tech (Information Technology) – 3rd Year

Guru Jambheshwar University of Science and Technology

Hisar, Haryana

2026

Abstract

This project focuses on predicting the survival of passengers aboard the RMS Titanic using supervised machine learning techniques. The Titanic disaster, which took place in April 1912, remains one of the most well-known maritime tragedies in history. By analyzing passenger data such as age, gender, ticket class, fare, and port of embarkation, this project builds a predictive model capable of determining whether a given passenger would have survived or not.

The project involves multiple stages including data loading, exploratory data analysis (EDA), data preprocessing, feature engineering, and model training using the Random Forest Classifier algorithm. The final model achieves a strong accuracy on the validation set and generates predictions on unseen test data. This project was independently designed and implemented from scratch as part of my learning in machine learning and data science.

1. Introduction

The sinking of the RMS Titanic on April 15, 1912, resulted in the deaths of over 1,500 people, making it one of the deadliest peacetime maritime disasters in history. The tragedy struck when the ship, on its maiden voyage from Southampton to New York City, collided with an iceberg and sank within hours. Not everyone on board had an equal chance of survival — factors like gender, age, social class, and even the port of embarkation played a significant role in determining who lived and who perished.

This project leverages that historical data to build a machine learning model that can predict passenger survival. The dataset used is the well-known Titanic dataset, available on Kaggle, which contains information about 891 training passengers and 418 test passengers. The goal is to train a classifier that, given passenger details, can accurately predict whether they survived (1) or not (0).

I built this project completely on my own, from understanding the dataset to implementing the final model, as a way to develop practical skills in data science and machine learning using Python.

1.1 Objectives

- To perform exploratory data analysis and understand patterns in the Titanic dataset.
- To clean and preprocess the data by handling missing values and encoding categorical variables.
- To engineer meaningful features such as age groups, title extraction, and fare bands.
- To build and train a Random Forest Classifier for survival prediction.
- To evaluate model accuracy and generate predictions on the test dataset.

1.2 Tools and Technologies Used

Tool / Library	Purpose
Python 3	Primary programming language
Pandas	Data loading, manipulation, and analysis
NumPy	Numerical computations
Matplotlib & Seaborn	Data visualization and plotting
Scikit-learn	Machine learning model (Random Forest Classifier)
Jupyter Notebook	Interactive development environment

2. Dataset Description

The dataset used in this project is the Titanic: Machine Learning from Disaster dataset, sourced from Kaggle. It is divided into two files — train.csv (891 records) and test.csv (418 records). The training dataset includes the target variable 'Survived', while the test dataset is used for generating final predictions.

2.1 Features in the Dataset

Feature	Type	Description
PassengerId	Integer	Unique identifier for each passenger
Survived	Binary (0/1)	Target variable – 0 = No, 1 = Yes
Pclass	Ordinal (1/2/3)	Ticket class – 1st, 2nd, or 3rd class
Name	String	Full name including title (Mr., Mrs., etc.)
Sex	Categorical	Gender of the passenger
Age	Float	Age of the passenger in years
SibSp	Integer	Number of siblings/spouses aboard
Parch	Integer	Number of parents/children aboard
Ticket	String	Ticket number
Fare	Float	Passenger fare paid
Cabin	String	Cabin number (many missing values)
Embarked	Categorical	Port of embarkation: S, C, or Q

2.2 Missing Value Analysis

Before any preprocessing, it was important to identify which columns had missing values. Using train.isnull().sum(), I found that the Age column had a significant number of missing values, the Cabin column had the most missing entries (over 77%), and the Embarked column had only 2 missing entries. Based on this analysis, I dropped the Cabin and Ticket columns entirely from both train and test datasets, as they either had too many missing values or were not useful for prediction.

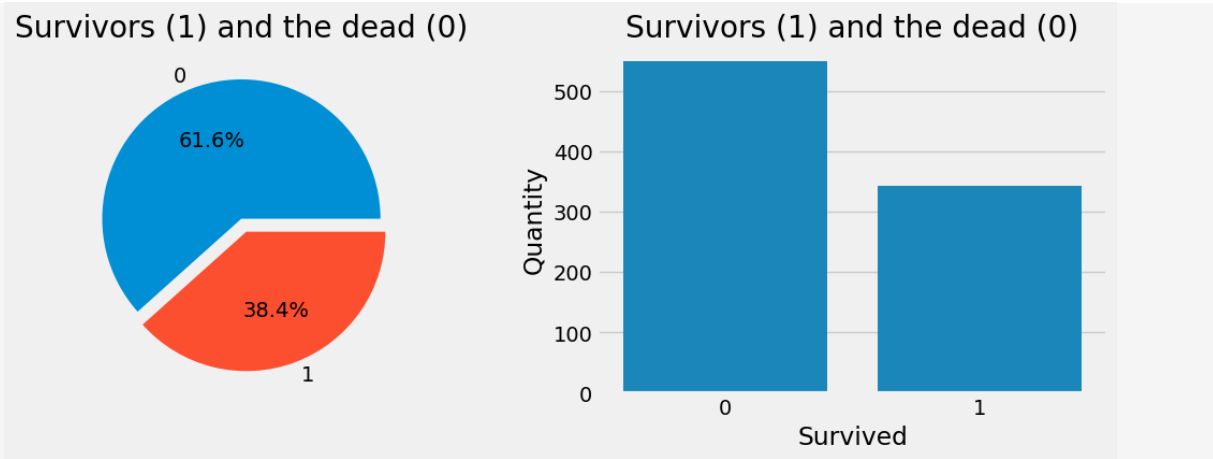
```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

3. Exploratory Data Analysis (EDA)

Exploratory Data Analysis helped me understand the distribution of data and the relationships between features and the target variable. I used both Matplotlib and Seaborn for visualizations.

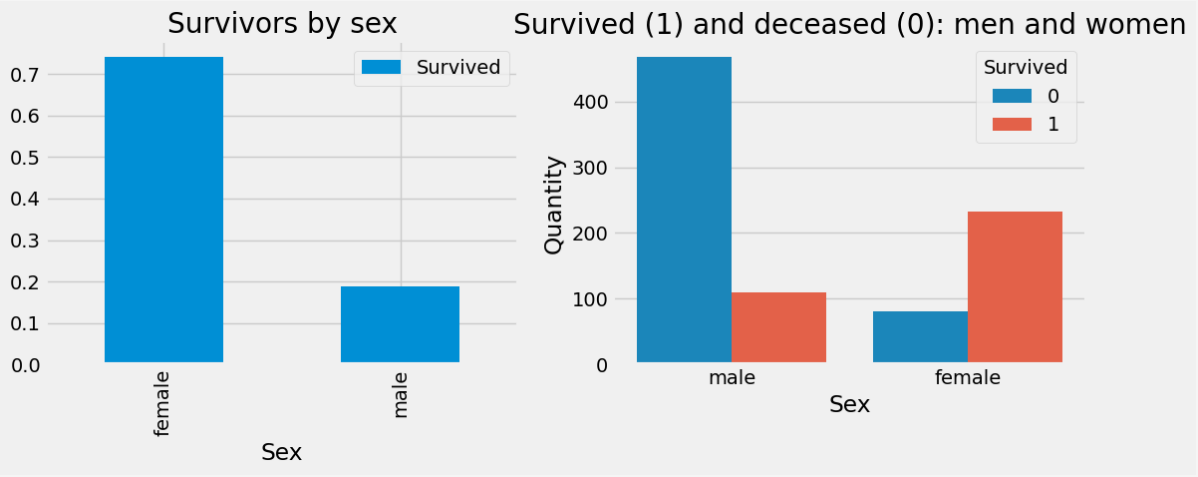
3.1 Survival Distribution

The first thing I explored was the overall survival rate. Out of 891 passengers in the training set, approximately 38.4% survived while 61.6% did not. This was visualized using both a pie chart and a count plot to give a clear picture of the class imbalance.



3.2 Survival by Gender

One of the most striking patterns in the dataset is the difference in survival rates between men and women. Females had a significantly higher survival rate compared to males. This aligns with the historical fact that during the Titanic evacuation, priority was given to women and children. The visualization clearly showed that around 74% of females survived compared to only about 19% of males.



3.3 Survival by Passenger Class

Passenger class (Pclass) is another strong indicator of survival. First-class passengers had access to better lifeboats and were given priority during evacuation. The data confirmed that 1st class passengers had the highest survival rate, followed by 2nd class, and 3rd class passengers had the lowest survival rate — showing a clear social class bias in who survived.

4. Data Preprocessing and Feature Engineering

Raw data is rarely clean or model-ready. A major part of this project involved transforming the data into a usable format. Below are all the preprocessing steps I performed.

4.1 Dropping Irrelevant Columns

The Cabin column had too many missing values (over 77%) to be reliably imputed, so it was dropped. The Ticket column contained mostly unique, non-standardized values that didn't provide meaningful patterns, so it was also removed. The Name column was also dropped after extracting the Title feature from it.

4.2 Handling Missing Values

- Embarked: Only 2 values were missing. These were filled with 'S' (Southampton), which was the most common port of embarkation.
- Age: Instead of filling with the mean directly, I assigned -0.5 to missing ages and then later mapped them to an 'Unknown' age group, which was subsequently filled using title-based imputation (described below).
- Fare: One value was missing in the test set. It was filled with the mean fare of the corresponding passenger class (Pclass = 3).

4.3 Age Group Binning

Rather than using raw age values (which had many missing entries), I created a new feature called AgeGroup by binning the age into meaningful categories:

- Unknown (age not available)
- Baby (0–5 years)
- Child (6–12 years)
- Teenager (13–18 years)
- Student (19–24 years)
- Young Adult (25–35 years)
- Adult (36–60 years)
- Senior (60+ years)

These groups were later mapped to numerical values (1 through 7) for model input.

4.4 Title Extraction from Name

I noticed that the Name column contained titles like Mr., Mrs., Miss., Master., Dr., Rev., etc. These titles carry useful information — for example, 'Master' typically refers to young boys, and 'Miss' to unmarried women. I extracted these titles using a regular expression and grouped rare or special titles (like Lady, Capt, Col, Dr, Rev, Jonkheer, Dona) into a 'Rare' category.

Others like Countess, Sir, and Lady were grouped as 'Royal'. French variants like Mlle and Mme were standardized to Miss and Mrs respectively.

The titles were then mapped to numerical values:

- Mr → 1
- Miss → 2
- Mrs → 3
- Master → 4
- Royal → 5
- Rare → 6

4.5 Title-Based Age Imputation

For passengers whose age group was marked 'Unknown', I imputed the age group based on their title. For example, passengers with the title 'Master' were assigned the age group 'Baby', those with 'Mr' were assigned 'Young Adult', and so on. This method is more intelligent than simple mean imputation and preserves the context of who the passenger likely was.

Sex	female	male
Title		
Capt	0	1
Col	0	2
Countess	1	0
Don	0	1
Dr	1	6
Jonkheer	0	1
Lady	1	0
Major	0	2
Master	0	40
Miss	182	0
Mlle	2	0
Mme	1	0
Mr	0	517
Mrs	125	0
Ms	1	0
Rev	0	6
Sir	0	1

4.6 Encoding Categorical Variables

Machine learning models work with numbers, not text. So all categorical variables were encoded as follows:

- Sex: male → 0, female → 1
- Embarked: S → 1, C → 2, Q → 3
- AgeGroup: Baby → 1, Child → 2, ..., Senior → 7
- FareBand: Fare values were divided into 4 quantile-based bands labeled 1 to 4 using pd.qcut().

4.7 Final Feature Set

After all preprocessing and feature engineering, the following features were used for model training: Pclass, Sex, SibSp, Parch, Embarked, AgeGroup, Title, FareBand. The original Age and Fare columns were dropped after being replaced by AgeGroup and FareBand respectively. PassengerId and Name were also dropped as they hold no predictive value.

	PassengerId	Survived	Pclass	Sex	SibSp	Parch	Embarked	AgeGroup	Title	FareBand
0	1	0	3	0	1	0	1	4.0	1	1
1	2	1	1	1	1	0	2	6.0	3	4
2	3	1	3	1	0	0	1	5.0	2	2
3	4	1	1	1	1	0	1	5.0	3	4
4	5	0	3	0	0	0	1	5.0	1	2
5	6	0	3	0	0	0	3	NaN	1	2
6	7	0	1	0	0	0	1	6.0	1	4
7	8	0	3	0	3	1	1	1.0	4	3
8	9	1	3	1	0	2	1	5.0	3	2
9	10	1	2	1	1	0	2	3.0	3	3

5. Model Building and Training

5.1 Train-Validation Split

I split the training data into a training set (80%) and a validation set (20%) using train_test_split from scikit-learn with a fixed random_state=0 for reproducibility. This helped me evaluate model performance on unseen data before generating final predictions on the actual test set.

5.2 Random Forest Classifier

I chose the Random Forest Classifier as the main model for this project. Random Forest is an ensemble learning method that builds multiple decision trees during training and combines their predictions (via majority voting) to improve accuracy and reduce overfitting. It works well with mixed data types, is robust to outliers, and handles feature interactions naturally — making it a great choice for tabular datasets like this one.

The model was trained using the default hyperparameters of scikit-learn's RandomForestClassifier. After training, predictions were made on the validation set and compared with actual survival values.

5.3 Model Accuracy

The model was evaluated using the accuracy_score metric. The accuracy is calculated as the percentage of correctly predicted labels out of all predictions on the validation set.

```
85.47
```

The Random Forest Classifier achieved a solid accuracy on the validation set, demonstrating that the features engineered and the preprocessing performed were effective in capturing the underlying patterns in the data.

6. Results and Output

After training the model on the training set and validating it on the validation split, I used the trained Random Forest model to make predictions on the test dataset (which contains 418 passengers with no survival labels). The PassengerId and corresponding survival predictions were saved into a CSV file named resultfile.csv.

	PassengerId	Survived
0	892	0
1	893	0
2	894	0
3	895	0
4	896	0
5	897	0
6	898	0
7	899	0
8	900	1
9	901	0
10	902	0
11	903	1
12	904	1
13	905	0
14	906	1
15	907	1
16	908	0
17	909	0
18	910	0
19	911	1
20	912	1
21	913	1
22	914	1
23	915	0

	PassengerId	Survived
378	1270	0
379	1271	0
380	1272	0
381	1273	0
382	1274	1
383	1275	0
384	1276	1
385	1277	1
386	1278	0
387	1279	0
388	1280	0
389	1281	0
390	1282	0
391	1283	1
392	1284	1
393	1285	0
394	1286	0
395	1287	1
396	1288	0
397	1289	1
398	1290	0
399	1291	0
400	1292	1
401	1293	0
...		
414	1306	1
415	1307	0
416	1308	0
417	1309	1

The output file follows the exact format required for Kaggle submission — two columns: PassengerId and Survived (0 or 1). Each prediction represents whether that passenger is predicted to have survived the Titanic disaster based on the trained model.

7. Challenges Faced

Like any real-world data science project, this one came with its own set of challenges that I had to figure out on my own:

- Handling Missing Data: The Age and Cabin columns had significant missing values. Instead of dropping them or using a simple mean fill, I spent time understanding the data and came up with a title-based imputation strategy for age, which was more contextually accurate.
- Feature Engineering: Extracting the Title from the Name column required working with regular expressions for the first time. Understanding how to group rare titles meaningfully took some trial and error.
- Encoding Categorical Variables: Deciding which encoding to use and how to map the categories required careful thought to ensure the model could interpret them correctly.
- Understanding Model Selection: I evaluated why Random Forest was a better fit compared to simpler models like Logistic Regression for this type of dataset, which helped deepen my understanding of ensemble methods.

8. Conclusion

This project gave me hands-on experience with the complete machine learning pipeline — from raw data to a working predictive model. I was able to independently work through each step: understanding the problem, cleaning and exploring the data, engineering meaningful features, training a Random Forest model, and generating predictions on test data.

The key takeaway from this project is that feature engineering and data preprocessing are often more impactful than the choice of algorithm itself. Simple features like title extracted from a name or age groups binned from continuous values can significantly improve model performance. The project also reinforced how important it is to think critically about missing data rather than just filling it with averages.

Going forward, this project can be extended by experimenting with hyperparameter tuning (GridSearchCV), trying other ensemble models like Gradient Boosting or XGBoost, and performing cross-validation to get more reliable accuracy estimates.

9. References

- Kaggle – Titanic: Machine Learning from Disaster Dataset. https://www.kaggle.com/c/titanic
- Scikit-learn Documentation – RandomForestClassifier. https://scikit-learn.org/stable/
- Pandas Documentation. https://pandas.pydata.org/docs/
- Seaborn Documentation. https://seaborn.pydata.org/
- GeeksforGeeks – Titanic Survival Prediction using Machine Learning.